

Turtle ID

Kaplumbaga Fotograf Tanim Sistemi

Clean Code | SOLID | Multi-Agent | Odev Raporu

Proje Adi:	Turtle ID — Kaplumbaga Fotograf Tanim Sistemi
Ogrenci:	Yusuf Altunbay
GitHub:	github.com/YusufAltunbay/HW_Turtle
Programlama Dili:	Python 3.11+
Kullanici Arayuzu:	PySide6 (Qt 6)
Veritabani:	SQLite + SQLAlchemy

3.974

Satir Kod

6

Ajan

41

Test

21

Git Commit

%100

Test Basarisi

ICERIK

- Projenin Amaci ve Genel Yapisi
- Kullanilan Teknolojiler
- Coklu Ajan (Multi-Agent) Mimarisi
- SOLID Prensipleri
- Clean Code Prensipleri
- Temel Is Akislari
- Test Sonuclari
- Sonuc ve Degerlendirme

1. Projenin Amaci ve Genel Yapisi

Bu proje, deniz kaplumbagalarini fotograftan taniyan ve kayit altina alan bir yazilim sistemidir. Temel problem sudur: farkli zamanlarda, farkli isik kosullarinda ya da farkli acidan cekilen iki fotograf, ayni kaplumbagaya mi ait yoksa farkli bireylere mi? Bu soruyu yapayzeka destekli goruntu isleme teknikleriyle cevaplar.

Sistem iki temel islevi yerine getirir. Birincisi kayit: bir kaplumbaganin fotografini sisteme yukleyerek isim ve tur bilgisiyle birlikte veritabanina kaydeder. Ikincisi dogrulama: yeni cekilen bir fotografın, daha once kaydedilmis kaplumbagalardan hangisine ait oldugunu benzerlik skoru uzerinden tespit eder.

Nasil Calisir?

EfficientNet-B0 adli derin ogrenme modeli, her fotograftan 1280 sayidan olusan bir parmak izi (embedding vektörü) cikarir. Bu sayilar, kaplumbaganin kabuk deseni, rengi ve genel gorunumunu temsil eder. Iki fotografın parmak izleri birbirine ne kadar benziyorsa, benzerlik skoru o kadar yuksek cikar (0.0 ile 1.0 arasinda). Varsayilan esik 0.82'dir; bu degerin uzerinde esleme basarili sayilir.

Ornek Senaryo

Arastirmaci sabah sahile gelen bir kaplumbagayi fotograflar ve sisteme kaydeder. Uc ay sonra benzer bir kaplumbaga tekrar gorulur. Yeni fotograf sisteme yuklenir ve sistem '% 91 benzerlik — Alpha' diyerek bunun ayni birey oldugunu tespit eder.

Mimari Ozet

Proje, Clean Architecture prensipleriyle katmanlara ayrilmistir. Her katman yalnızca bir alttakini tanir; yukaridakilere bagimli degildir. Bu sayede herhangi bir parcayi degistirmek tum sistemi etkilemez.

```
Kullanici Arayuzu (PySide6 – gorsel pencereler)
|
Is Akislari (Kayit ve Dogrulama mantigi)
|
Ajanlar (her biri tek bir isi yapar)
|
Altyapi / DB (SQLite, EfficientNet-B0, OpenCV)
```

2. Kullanılan Teknolojiler

Teknoloji	Kullanım Amacı
Python 3.11+	Ana programlama dili
PySide6 (Qt 6)	Masaüstü kullanıcı arayüzü
EfficientNet-B0 (timm)	Fotografтан 1280 boyutlu özellik vektörü çıkarma
OpenCV	Görüntü ön işleme: yeniden boyutlandırma, normalize
Cosine Similarity	İki vektörü karşılaştırarak benzerlik skoru üretme
SQLite + SQLAlchemy	Kaplımbaga ve embedding verilerini saklama
EventBus (özel)	Ajanlar arası mesajlaşma altyapısı
pytest	Otomatik test (41 test, %100 geçme)
loguru	Renkli ve dosyaya yazılabilen log sistemi

Proje Dizin Yapısı

```
HW_Turtle/  
  src/turtle_id/  
    agents/      -> 6 ajan (her biri ayrı sorumlu)  
    core/  
      models/    -> Turtle, Embedding, MatchResult (saf veri nesneleri)  
      ports/     -> Soyut arayüzler (DIP için)  
      use_cases/ -> Kayıt ve doğrulama iş akışları  
  infrastructure/  
    vision/      -> EfficientNet-B0, OpenCV ön işleme  
    matching/    -> Cosine benzerliği  
    persistence/ -> SQLite veritabanı  
    ui/          -> PySide6 ekranlar  
    shared/      -> EventBus, EventType  
    container.py -> Tüm bağımlılıklar burada bir araya gelir  
  tests/        -> 41 otomatik test  
  data/         -> Veritabanı ve test fotoğrafları  
  run.bat       -> Çift tıklayarak başlat
```

3. Coklu Ajan (Multi-Agent) Mimarisi

Sistem, her biri tek bir gorevi olan 6 ajandan olusur. Ajanlar birbirini dogrudan cagirmaz; mesajlarini EventBus uzerinden yayimlar. Bu yaklasim, bir ajandaki degisikligin diger ajanlari etkilemesini onler.

Ajan	Gorevi
ConfigAgent	Uygulama ayarlarini (esik, top-k) okur ve yazar
PhotoValidationAgent	Fotografin net, aydinlik ve gecerli formatda oldugunu kontrol eder
ImageAgent	Fotograftan EfficientNet-B0 ile embedding vektörü üretir
MatchingAgent	Sorgu vektorunu veritabanindaki vektorlerle karsilastirir
DataAgent	Kaplumbaga kayıt, güncelleme, listeleme işlemlerini yapar
UIAgent	PySide6 ekranlarini yönetir, buton tiklamalarini is akislarina iletir

Ajanlar Nasil Haberlesir?

Ajanlar arasi iletisim EventBus adli bir yayim/abone sistemiyle saglanir. Bir ajan islemi bitirdiginde mesaj yayimlar; ilgilenen diger ajanlar bu mesaji dinler ve gereğini yapar. Boylece ajanlar birbirini tanımak zorunda kalmaz, bu da sistemi daha kolay test edilebilir ve genisletilebilir kilar.

```
# Ornek: Esleme tamamlandiginda dogrulama logu otomatik kaydediliyor

class MatchingAgent:
    def match(self, sorgu_vektoru):
        sonuc = self.matcher.esles(...)
        self.yayimla(ESLEME_BULUNDU, sonuc) # mesaji yayimla

class DataAgent:
    def baslarken(self):
        self.dinle(ESLEME_BULUNDU, self.log_kaydet) # mesaji dinle

    def log_kaydet(self, sonuc):
        # MatchingAgent'i hic tanimadan logu yazar
        veritabani.kaydet(sonuc)
```

Ajan Yasam Dongusu

Her ajan BaseAgent adli temel siniftan turetilir. Bu sinif; baslat, durdur ve saglik kontrol gibi ortak islevleri saglar. Her ajan kendi ozel gorevini _register_handlers() metodunda tanimlar.

4. SOLID Prensipleri

SOLID, yazılım tasarımıda beş temel prensibi ifade eder. Bu projede her biri bilerek uygulanmıştır.

S — Tek Sorumluluk (Single Responsibility)

Her sınıf yalnızca bir işi yapar. Örneğin PhotoValidationAgent sadece fotoğrafın kalitesini kontrol eder; veritabanına yazmaz, eşleme yapmaz. DataAgent sadece kayıt işlemleri yapar; görüntü işlemez.

```
# PhotoValidationAgent – tek görevi fotoğrafı kontrol etmek:
def validate(self, fotograf_yolu):
    self._dosya_var_mi_kontrol(yol)          # Dosya mevcut mu?
    self._uzanti_kontrolu(yol)              # .jpg, .png vs.
    self._bulaniklik_kontrolu(goruntu)      # Laplacian skoru > 50
    self._parlaklik_kontrolu(goruntu)       # Çok karanlık/acık değil
    return ValidationResult(...)
```

O — Açık/Kapalı (Open/Closed)

Mevcut kodu değiştirmeden yeni özellik eklenebilir. Örneğin cosine benzerliği yerine FAISS algoritması kullanmak istenseydi, sadece yeni bir sınıf yazılıp container.py'da tek satırda takilirdi.

```
# Mevcut:
eslestirici = CosineMatcher()

# Yeni algoritma eklemek için sadece şu değişir:
eslestirici = FAISSMatcher() # Diğer hiç bir kod değişmez
```

L — Liskov Yerine Geçme (Liskov Substitution)

Tüm ajanlar BaseAgent sınıfını doğru biçimiyle uygular. Bu sayede ajanlar bir liste içerisinde birbirleri yerine kullanılabilir ve sistem bunları aynı şekilde yönetebilir.

```
for ajan in [validation_agent, image_agent, matching_agent, data_agent]:
    ajan.baslat()          # Her biri aynı sözleşmeye uyar
    ajan.saglik_kontrol()
    ajan.durdur()
```

I — Arayüz Ayırma (Interface Segregation)

Her arayüz yalnızca ihtiyaç duyulan metodları içerir. Örneğin IImagePreprocessor sadece preprocess() metoduna sahiptir. Hiç kullanılmayacak metod tanımlamak zorunda kalinmaz.

D — Bağımlılık Tersine Çevrilmesi (Dependency Inversion)

Ajanlar somut sınıflar yerine soyut arayüzlere bağımlıdır. Bu sayede gerçek model yerine test sırasında sahte bir nesne enjekte edilebilir; testler hızlanır ve bağımsız olur.

```
# Kotü: Ajan somut sinifi dogrudan olusturuyor
class ImageAgent:
    def __init__(self):
        self.model = TimmEfficientNetExtractor() # Degistirilmesi zor

# İyi: Arayuz disaridan enjekte ediliyor (DIP)
class ImageAgent:
    def __init__(self, model: IEmbeddingExtractor):
        self.model = model # Mock da, gercek model de olabilir
```

5. Clean Code Prensipleri

Anlamli Isimler

Her sinif, metod ve degisken ismi ne yaptigini dogrudan anlatir. Kisaltma veya belirsiz isimler kullanilmaz.

```
# Sinif isimleri ne yaptiklarini anlatir:
PhotoValidationAgent      # Fotograflari dogrular
CosineMatcher              # Cosine benzerligi ile eslestirir
SQLiteTurtleRepository    # SQLite üzerinden kaplumbaga verileri
RegisterTurtleUseCase     # Kaplumbaga kayit is akisi

# Metod isimleri acik ve anlasilidir:
validate(fotograf_yolu)   # Dogrular
find_best_match(vektor)   # En iyi eslemeyi bulur
has_embeddings()          # "Embedding var mi?" sorusunu sorar
is_normalized()           # "Normalize mi?" sorusunu sorar
```

Kucuk ve Tek Amacli Fonksiyonlar

Her fonksiyon tek bir isi yapar. Uzun bir fonksiyon yerine, her biri kendi isini yapan kucuk fonksiyonlar tercih edilir. Bu hem okunabilirlik hem de test edilebilirlik acisinden avantajlidir.

```
# Kotu: Her seyi yapan buyuk bir fonksiyon
def isle(yol):
    if not os.path.exists(yol): return False
    if yol.split('.')[ -1] not in [...]: return False
    img = cv2.imread(yol)
    if img.shape[0] < 100: return False
    # ... 80 satir devam ediyor

# Iyi: Her kontrolun kendi kucuk fonksiyonu var
def validate(self, yol):
    self._dosya_kontrolu(yol, sonuc)
    self._uzanti_kontrolu(yol, sonuc)
    self._boyut_kontrolu(goruntu, sonuc)
    self._bulaniklik_kontrolu(goruntu, sonuc)
    return sonuc
```

Hatalar Exception Yerine DTO ile Tasinir

Is akislari hata durumunda exception firlatmak yerine, basarili/basarisiz bilgisini icerek bir sonuc nesnesi doner. Bu, kullanıcı arayuzunun hataya zarif bir sekilde tepki vermesini saglar.

```
def execute(self, istek):
    dogrulama = self.validation_agent.validate(istek.fotograf_yolu)

    if not dogrulama.is_valid:
        return KayitSonucu(
            basarili=False,
            hatalar=dogrulama.hatalar      # Hata mesajlari UI'a tasiniyor
        )

    # ... devam
```

Hata Izolasyonu

EventBus'ta bir abonenin hatasi diger aboneleri durdurmaz. Her abone ayri bir try-except blogunda cagrilir.

```
for abone in aboneler:
    try:
        abone(olay_turu, veri)
    except Exception as hata:
        logger.error(f"Abone hatasi: {hata}")
        # Diger aboneler calismaya devam eder
```


6. Temel İş Akışları

Kaplumbaga Kayıt Akışı

Kullanıcı arayüzünde fotoğraf seçilip isim girildiğinde aşağıdaki adımlar arka planda (QThread ile) çalışır:

- Fotoğraf geçerliliği kontrol edilir (net, aydınlık, doğru format)
- Geçerliyse EfficientNet-B0 modeli fotoğraftan 1280 sayılı bir vektör üretir
- Kaplumbaga bilgileri (isim, tür, vektör) veritabanına kaydedilir
- Kullanıcıya başarı mesajı gösterilir

```
Kullanıcı 'Kaydet' tıklar
|
+--> Fotoğraf kontrolü          [net mi, boyut uygun mu?]
|
+--> EfficientNet-B0           [1280 sayılı parmak izi]
|
+--> Veritabanına kaydet       [TurtleORM + EmbeddingORM]
|
--> Başarı mesajı
```

Kaplumbaga Doğrulama Akışı

Yeni fotoğraf yüklendikten sonra sistem, veritabanındaki tüm kayıtlı kaplumbagalarla karşılaştırma yapar:

- Yeni fotoğraftan embedding vektörü üretilir
- Bu vektör, veritabanındaki tüm vektörlerle cosine benzerliği hesaplanır
- En yüksek skoru alan kaplumbaga aday gösterilir
- Skor esigi (0.82) geçerse 'Eşleme Bulundu', geçmezse 'Bulunamadı' yazılır
- Her doğrulama otomatik olarak geçmiş loguna yazılır

```
Kullanıcı 'Doğrula' tıklar
|
+--> Fotoğraf kontrolü
|
+--> Yeni fotoğraftan vektör üret
|
+--> Tüm kayıtlı vektörlerle karşılaştır [cosine similarity]
|
+--> En iyi adayı seç
|
+--> Skor >= 0.82?
      Evet --> 'Eşleme Bulundu – Fotoğraf: Alpha, Benzerlik: %91'
      Hayır -> 'Eşleme Bulunamadı'
```

Audit Trail (Dogrulama Gecmisi)

Her dogrulama islemi otomatik olarak verification_log tablosuna kaydedilir.

Hangi fotograf, kac puanla, hangi kaplumbagayla eslesti bilgisi saklanir.

Bu islem tamamen EventBus uzerinden gerceklesir; is akisi bunu bilmez.

7. Test Sonuclari

Proje, 41 otomatik testle dogrulanmistir. Testler iki kategoridedir: birim testleri her sinifi ayri ayri test ederken, entegrasyon testleri veritabani islemlerini gercek (bellek ici) SQLite üzerinde calistirir.

Test Dosyasi	Kapsam	Test Sayisi
test_event_bus.py	Pub/sub, thread guvenlik, hata izolasyonu	7
test_matcher.py	Cosine esleme, esik mantigi, bos liste	7
test_models.py	Turtle, Embedding, MatchResult nesneleri	13
test_validation_agent.py	Fotograf dogrulama ve EventBus entegrasyonu	4
test_repository.py	Veritabani CRUD, vektor saklama, silme	10
TOPLAM		41 / 41
		GECTI

Ornek Test

```
# Ayni vektorun kendisiyle eslesmesi 1.0 olmalı:
def test_ayni_vektor_eslesmeli(matcher):
    vektor = rastgele_normalize_vektor(128)
    sonuc = matcher.find_best_match(
        vektor, [Embedding(vector=vektor)], threshold=0.9
    )
    assert sonuc.is_match == True
    assert sonuc.similarity_score == pytest.approx(1.0, abs=1e-5)

# Bos liste eslememeli:
def test_bos_liste_eslememeli(matcher):
    sonuc = matcher.find_best_match(vektor, aday_listesi=[], threshold=0.75)
    assert sonuc.is_match == False
```

Gercek Fotograf Uzerinde Pipeline Testi

Sentetik fotoğraflar üzerinde yapılan pipeline testinde:

- Ayni kaplumbaganin karanlik fotografiyla: 0.875 (esleme)
- Ayni kaplumbaganin parlak fotografiyla: 0.962 (esleme)
- Farkli kaplumbagayla karsilastirma: 0.414 (esleme yok)

Sonuc: Hicbir yanlis pozitif esleme gozlemlenmedi (0 / 10 karsilan karsilastirmada).

8. Sonuc ve Degerlendirme

Bu projede, kaplumbaga tanima problemini cozecek tam islevli bir masaustu uygulaması gelistirilmistir. Proje boyunca yazilim muhendisligi prensipleri bilerek ve tutarli sekilde uygulanmaya calisilmistir.

Kazanimler

- SOLID prensipleri somut proje ornekleriyle uygulanmistir. Ornegin adapter degistirmek icin yalnızca container.py'da tek satirlik degisiklik yeterlidir (OCP).
- Clean Code anlayisiyla her sinif ve metod tek bir isi yapacak sekilde tasarlanmistir; anlamlı isimler, kucuk fonksiyonlar ve hata izolasyonu ozenle uygulanmistir.
- Multi-agent mimarisi, sistemi bagimsiz parcalara ayirarak hem test edilebilirlik hem de genisletilebilirlik saglamistir.
- EventBus mimarisi sayesinde ajanlar birbirini import etmeden haberlesir; bu da coupling'i en aza indirir.
- 41 otomatik test yazilmis ve %100 gecme oranı saglanmistir. Testler hizli calisir (4 saniye) cunku veritabanı bellekte tutulur.

Gozlemler ve Sinirlamalar

- EfficientNet-B0 genel amacli bir modeldir; kaplumbaga ozelinde egitilmemistir. Gercek fotograflarda esik degerinin 0.60-0.65'e dusurulmesi daha iyi sonuc verebilir.
- Sistem su an tum embeddingleri RAM'e yukleyerek karsilastirma yaptiginden, cok buyuk veritabanlarında FAISS gibi hizli yaklasik arama algoritması kullanmak gerekebilir.

Proje Buyuklugu

Metrik	Deger
Toplam kaynak kodu	3.974 satir
Test kodu	493 satir
Ajan sayısı	6
Veritabanı tablosu	3 (turtles, embeddings, verification_log)
UI sayfası	5
Git commit sayısı	22
Test basari oranı	%100 (41/41)

github.com/YusufAltunbay/HW_Turtle